

Linux kernel for real-time applications while retaining the general-purpose capabilities of Linux.

The main features introduced by the PREEMPT_RT patch are listed in Table 1.

When the PREEMPT_RT patch is applied to the Linux kernel and booted, the real-time features become available. Real-time tasks can be created, and use the real-time scheduling classes to achieve deterministic and timely execution.

To use real-time capabilities, you need to configure the applications to run with elevated privileges (often as root) since real-time tasks have higher scheduling priorities and require special permissions.

PREEMPT_RT patch significantly improves real-time performance; it may not be suitable for certain ultra-high-performance real-time applications where absolute timing guarantees are critical. This patch is an external addition to the Linux kernel and may not always be in sync with the latest official kernel releases. Therefore, it is crucial to carefully select the kernel version and the corresponding PREEMPT_RT patch version that are compatible and well-maintained for your application needs. Additionally, the kernel configuration options and tuning for real-time performance should be done with care and understanding of specific requirements.

Cyclictest

We take a look at a test conducted in the PREEMPT_RT patch, using the Cyclictest command. Cyclictest is a command-line utility in Linux, often used for measuring real-time performance and latency of the system. It is particularly helpful for testing and analysing the Linux kernel's ability to handle periodic tasks with strict timing requirements.

To use Cyclictest, you need to install it on your Linux system. It is often available in most distribution repositories. If it is not installed already, use the package manager specific to your distribution. For example, on Ubuntu and Debian-based systems, the following command can be used to

Feature	Description
Preemption	- Enhances the kernel's ability to be preempted at more points during its execution - This allows higher-priority tasks to interrupt lower-priority ones more quickly, reducing latencies and improving responsiveness
Kernel preemption	- Introduces kernel preemption, which allows high-priority kernel threads to preempt lower-priority ones - Improves the responsiveness of the kernel to real-time events
Priority inheritance	- Implements priority inheritance protocols to prevent priority inversion, a situation where a higher-priority task is blocked by a lower-priority task holding a shared resource - Priority inheritance ensures that the resource-holding task temporarily inherits the priority of the blocked task, preventing priority inversions
High-resolution timers	Provides support for high-resolution timers, which allow applications to set timers with finer granularity, making it easier to meet tight timing constraints
Real-time scheduling	- Enhances the Linux scheduler to support real-time scheduling processes, such as SCHED_FIFO and SCHED_RR in addition to the standard CFS scheduler - These policies allow developers to create real-time tasks with strict timing requirements
Throttle mechanism	Introduces throttle mechanism to limit the execution time of low-priority tasks, preventing them from excessively delaying high-priority real-time tasks
Interrupt handling	- The patch optimises interrupt handling to reduce the latency associated with servicing interrupts - It reduces the time spent in interrupt context, making the system more responsive to real-time events
Soft and hard real-time support	- PREEMPT_RT patch offers both soft and hard real-time capabilities - Soft real-time systems aim to meet timing constraints most of the time but allow occasional deadline misses. Hard real-time systems must meet timing constraints strictly, without any misses
Kernel locking and synchronisation	To ensure data consistency and prevent race conditions, the patch addresses kernel locking and synchronisation mechanisms to handle concurrent access to shared data

Table 1: PREEMPT_RT patch features